Project Proposal: Parallel LLM Inference with Fault-Tolerant Pipeline

October 9, 2026 · MD TAUFIQUE HUSSAIN

1. Project Overview

Abstract. We will split a small pretrained language model (GPT-2 small, 12 transformer layers) across two worker processes, so that each worker runs half of the layers and passes intermediate activations to the other over RPC. A coordinator will manage concurrent requests, keep per-request KV-cache state isolated, apply backpressure through bounded queues, and recover from worker crashes by restarting the worker and replaying the prompt. Stale results from before a crash are discarded using request IDs, step numbers and worker epochs. We will verify that the distributed system produces exactly the same tokens as single-process inference, including after a failure, and we will measure token latency, throughput, bytes transferred and recovery time.

2. Problem Statement and Motivation

Large language models often do not fit on one machine or one GPU, so production systems split a model's layers across several devices. This is called pipeline (model) parallelism.

Splitting a model turns one program into a distributed system. The parts must exchange data over a network, keep per-request state consistent, avoid being overloaded, and survive the failure of any part. A crash in the middle of text generation also destroys the KV cache, the saved attention state that makes generation fast.

This project studies those distributed-systems problems on a model small enough to run on ordinary CPUs. The core question: **can a two-stage inference pipeline give exactly the same output as a single process, under concurrency and worker failures, and what does the distribution cost in latency and network traffic?**

3. Objectives

1. Split GPT-2 small into two pipeline stages running in separate processes and exchanging activations over RPC.
2. Serve several requests concurrently while keeping each request's KV cache and token order isolated and correct.
3. Add bounded request queues, timeouts and backpressure so overload never causes unbounded memory growth.
4. Detect worker failures through heartbeats and timeouts, restart the worker, and rebuild its state by replaying the prompt.
5. Discard stale or duplicate outputs using request IDs, step numbers and worker epochs.
6. Optionally add micro-batching so both stages work at the same time on different requests.
7. Verify token-for-token equality with single-process inference, and measure latency, throughput, transferred bytes and recovery time.

4. Scope

In scope

- Inference only, with a supplied pretrained model (GPT-2 small, 124M parameters)
- Two pipeline stages in two separate OS processes, on one machine or two machines on a LAN
- Greedy decoding, so outputs are deterministic and comparable
- Concurrent requests, per-request KV caches, bounded queues, timeouts, heartbeats
- Crash recovery by worker restart and prompt replay
- Micro-batching as an optional stretch goal

Out of scope

- Training or fine-tuning the model
- Tensor parallelism (splitting a single layer across workers)
- More than two stages, GPU clusters, or models larger than about 350M parameters
- Coordinator fault tolerance (the coordinator is assumed reliable; this is stated as a failure assumption)
- Byzantine faults, security and authentication

5. System Architecture

The system has four components: clients, a coordinator, and two pipeline workers. Only the workers hold model weights; the coordinator holds the control logic for ordering, failure detection and recovery.

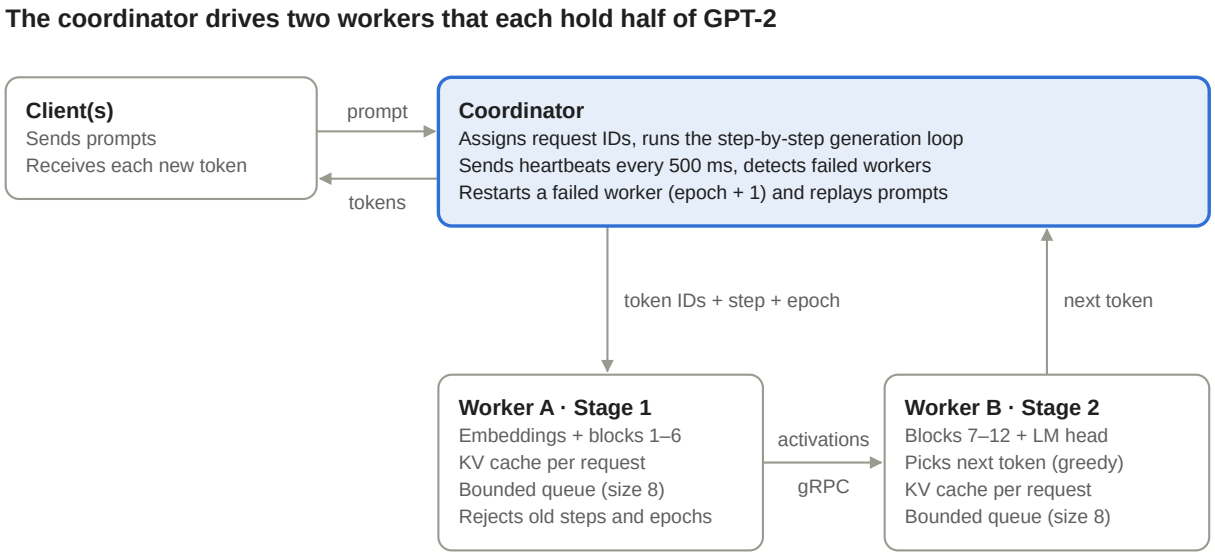


Figure 1. System architecture

Each generation step flows coordinator → Worker A → Worker B → coordinator, and the coordinator streams each new token back to the client. Putting all recovery logic in the coordinator keeps the workers simple and makes failures easier to reason about.

6. Detailed Design

6.1 Model split

Stage	Runs on	Contains	Output
Stage 1	Worker A	Token and position embeddings, transformer blocks 1–6	Hidden-state tensor, 768 values per token
Stage 2	Worker B	Transformer blocks 7–12, final layer norm, LM head	Logits, from which the next token is chosen greedily

6.2 Message format

Every RPC message carries these fields:

Field	Purpose
`request_id`	Unique ID of the user request (UUID)
`step`	Token position: 0 = prompt prefill, 1, 2, ... = each generated token
`epoch`	Incarnation number of the receiving worker; increases on every restart
`payload`	Token IDs (to Worker A) or activation bytes + shape + dtype (to Worker B)
`kind`	`PREFILL`, `DECODE`, `REPLAY`, `RELEASE`, or `HEARTBEAT`

Activations are sent as raw float32 bytes (`tensor.numpy().tobytes()`), which keeps messages small and makes byte counting exact.

6.3 Per-request KV cache

Each worker keeps a dictionary `kv_cache[request_id]` holding the cached keys and values for its own six layers only. An entry is created on `PREFILL`, extended on each `DECODE`, and deleted on `RELEASE` or after an idle timeout, so finished or abandoned requests do not leak memory. Two requests never share a cache entry.

6.4 Ordering and duplicate handling

Each worker tracks `expected_step[request_id]`.

- `step == expected` → process it, then increment the expected step.
- `step < expected` → stale duplicate (for example a retry that already succeeded): discard it and return the cached result if available.
- `step > expected` → a gap; reject it with `OUT_OF_ORDER` so the coordinator resends.
- `epoch` older than the worker's current epoch → message from before a restart: discard it.

The coordinator applies the same rule to results coming back, so a token for a given `(request_id, step)` is delivered to the client exactly once.

6.5 Bounded queues and backpressure

Each worker has an input queue with a fixed capacity (initially 8 items). When the queue is full, the worker answers `BUSY` instead of accepting work. The coordinator then waits with exponential backoff (10 ms, 20 ms, 40 ms, ...) before retrying, and limits the number of requests in flight. New client requests beyond a configured limit are rejected with a clear error rather than queued forever.

6.6 Failure detection

- Every RPC has a deadline (initially 2 s for decode steps, longer for prefill).
- The coordinator sends a heartbeat to each worker every 500 ms.
- A worker that misses 3 heartbeats in a row, or fails a call past its deadline, is declared failed.

6.7 Recovery by prompt replay

1. The coordinator stops sending new work to the failed stage and starts a fresh worker process with `epoch + 1`.
2. The new worker loads its half of the model weights from disk.
3. For every unfinished request, the coordinator sends a `REPLAY` message containing the original prompt plus all tokens generated so far. Both stages run this as a single prefill, rebuilding their KV caches in one pass.
4. Generation continues from the next step. Tokens already delivered to the client are never sent again.
5. Any late message tagged with the old epoch is discarded (section 6.4).

If only Worker B fails, Worker A's caches are still valid, but for simplicity both stages replay; this is noted as a possible optimization.

6.8 Micro-batching (stretch goal)

With several requests in flight, the coordinator keeps both stages busy: Worker A computes step *n* of request 2 while Worker B computes step *n* of request 1. If time allows, decode steps of up to 4 requests are grouped into one batch per stage to reduce per-message overhead.

6.9 Failure assumptions

- Crash-stop failures of workers only; a failed worker does not send wrong data.
- The coordinator is reliable (single point of failure, accepted for this project).
- Messages may be delayed, duplicated by retries, or lost, but are not corrupted.

7. Distributed Systems Concepts Addressed

Concept	Where it appears in this project
Model parallelism	Layers split across two worker processes (6.1)
Remote Procedure Call	Activations and tokens exchanged between processes via gRPC (6.2)
Pipelining	Both stages working on different requests at the same time (6.8)
Backpressure	Bounded queues, `BUSY` replies, backoff, in-flight limits (6.5)
Failure detection	Heartbeats and RPC deadlines (6.6)
Retries and idempotency	Duplicate steps discarded by `(request_id, step)` (6.4)
Fencing with epochs	Messages from a pre-crash worker rejected (6.4, 6.7)
Recovery by replay	KV cache rebuilt from the prompt and delivered tokens (6.7)
Communication overhead	Bytes and time spent on the network, measured per token (section 10)

8. Technology Stack and Resources

Need	Choice	Reason
Language	Python 3.11	Standard for ML work; one language for all components
Model	GPT-2 small (124M parameters, 12 layers)	Runs on a laptop CPU; simple, well-documented architecture
ML libraries	PyTorch, Hugging Face `transformers` (version pinned)	Load pretrained weights and access individual layers
RPC	gRPC with Protocol Buffers	Built-in deadlines, typed messages, widely used in industry
Process control	Python `subprocess` + a launcher script	Start, kill and restart workers for failure tests
Testing	`pytest`	Automated correctness and failure tests
Analysis	`pandas`, `matplotlib`	Benchmark tables and graphs
Version control	Git + GitHub	Code sharing and version history

Hardware. Two ordinary laptops or one laptop with at least 8 GB RAM and a 4-core CPU. GPT-2 small needs about 500 MB of memory per full copy, so each half fits easily. No GPU is required; a GPU is used only if available.

Cost. None; all software and the model are free and open source.

9. Correctness Verification Plan

The system is correct if it produces **exactly the same token IDs** as single-process GPT-2 with greedy decoding, for every prompt, under every test condition below.

#	Test	Pass condition
1	Reference: 20 prompts × 50 tokens on single-process GPT-2	Outputs saved as the ground truth
2	Same prompts through the two-stage pipeline, one at a time	Identical tokens to the reference
3	10 prompts sent concurrently	Each output identical to its own reference (no cache mixing)
4	Kill Worker B at a random step, then recover	Final output identical; no token delivered twice
5	Kill Worker A at a random step, then recover	Final output identical; no token delivered twice
6	Inject duplicate and old-epoch messages	All discarded; output unchanged
7	Flood the system beyond queue capacity	Clients get `BUSY` or rejection errors; no crash, memory stays bounded

All tests are automated with `pytest`, and crash tests use a script that sends `SIGKILL` to a worker at a chosen step. Both stages use float32 and the same library versions so tiny numeric differences cannot change the greedy choice.

10. Evaluation Metrics and Experiments

Metrics

- **Token latency:** time from one generated token to the next, reported as mean and p95
- **Time to first token:** time from request arrival to the first generated token
- **Throughput:** total tokens generated per second across all requests
- **Transferred bytes:** activation bytes sent between stages, per token and per request
- **Recovery time:** detection time + restart time + replay time after a crash
- **Duplicate work:** tokens recomputed during replay

Experiments

#	Experiment	Compared	Metrics
E1	Baseline vs. pipeline	Single process vs. two stages on one machine vs. two machines	Token latency, time to first token
E2	Concurrency scaling	1, 2, 4, 8 concurrent requests	Throughput, p95 latency
E3	Micro-batching (if built)	On vs. off at 4 and 8 requests	Throughput, worker idle time
E4	Communication cost	Prompt lengths 16, 64, 256 tokens	Bytes per prefill, bytes per decode step
E5	Failure recovery	Kill Worker A vs. Worker B at early, middle and late steps	Recovery time, duplicate work
E6	Overload	Arrival rate above capacity, queue sizes 4, 8, 16	Rejected requests, latency, memory use

Each experiment is repeated at least 5 times; results are reported as averages with p95 values. We expect the pipeline to be slower than single-process inference for a model this small, because network and serialization overhead dominate; measuring and explaining that overhead is a goal of the project, not a failure.

11. Deliverables

- ☐ Source code on GitHub: coordinator, two workers, client, gRPC definitions, launcher
- ☐ Automated test suite (correctness, concurrency, crash, duplicate and overload tests)
- ☐ Benchmark scripts and raw results
- ☐ Final report with design, results, graphs and analysis
- ☐ README with setup and run instructions
- ☐ Live demo: matching outputs, concurrent requests, a worker killed and recovered during generation

12. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
Hugging Face KV-cache format changes between library versions	Medium	High	Pin the `transformers` version on day 1; wrap cache handling in one module
Floating-point differences break exact token matching	Low	High	Greedy decoding, float32 on both stages, compare token IDs not raw values
Pipeline slower than single process	High	Low	Expected for a small model; report and explain the overhead
Recovery logic has subtle race conditions	Medium	High	Epoch fencing, step numbers, and repeated randomized crash tests
gRPC setup takes longer than planned	Low	Medium	Fall back to ZeroMQ, which has a simpler setup
Schedule slips	Medium	Medium	Micro-batching is optional and is dropped first

13. References

1. Radford et al., *Language Models are Unsupervised Multitask Learners* (GPT-2), OpenAI, 2019.
2. Huang et al., *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*, NeurIPS 2019.
3. Narayanan et al., *PipeDream: Generalized Pipeline Parallelism for DNN Training*, SOSP 2019.
4. Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM), SOSP 2023.
5. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.
6. Hugging Face `transformers` documentation, GPT-2 model and `past_key_values`.
7. gRPC documentation: deadlines, error codes and Python API.
8. Course project description, Topic 38: Parallel LLM Inference.